

=====

PROJECT 03

WEB APP SECURITY

Vulnerability Assessment & Professional Reporting

=====

TARGET:

[PortSwigger Web Security Academy](#)

VULNERABILITY:

[SQL Injection Authentication Bypass](#)

PREPARED BY:

Tushar Chaugule

DATE:

25/07/2026

VERSION:

1.0

=====

1.Executive Summary

During a security assessment of the PortSwigger Web Security Academy laboratory environment, a High-Severity SQL Injection vulnerability was identified within the application's authentication mechanism. Successful exploitation demonstrated that authentication could be bypassed, allowing unauthorized access to the administrator account without valid credentials. If present in a production environment, this vulnerability could result in unauthorized administrative access, exposure of sensitive information, and compromise of application integrity. Immediate remediation through secure query construction and parameterized statements is recommended.

2.Scope

Parameter	Details
Assessment Type	Black Box Web Application Testing
Target System	PortSwigger Web Security Academy
Environment	Educational Lab Environment
In-Scope Boundary	Target lab instance and primary authentication flow

Out-of-Scope	Any system, service, or infrastructure outside the designated lab
---------------------	---

3.Methodology

The security assessment followed a structured, manual testing approach aligned with the **OWASP Web Security Testing Guide (WSTG)** guidelines. The primary focus was on evaluating the application's authentication mechanisms and identifying potential input validation weaknesses leading to SQL Injection.

Workflow Overview

Reconnaissance -> Authentication Testing -> Input Validation -> SQL Injection Testing -> Verification -> Documentation

Step-by-Step Execution

1. Reconnaissance

- **Goal:** Map out the target application and identify entry points.
- **Actions:** Navigated to the application interface to locate user input fields, forms, and parameter endpoints—specifically the primary `/login` endpoint.

2. Authentication Testing

- **Goal:** Evaluate standard login behavior and credentials handling.
- **Actions:** Submitted standard test credentials (`admin / admin`) to map how the backend handles authentication requests. Observed generic error responses ("Invalid username or password"), confirming standard baseline behavior.

3. Input Validation

- **Goal:** Test input fields for improper sanitization or syntax escaping issues.
- **Actions:** Injected special SQL characters (specifically a single quote `'`) into the `Username` field. Received an `Internal Server`

Error (HTTP 500) from the backend, signaling an unhandled database syntax exception and confirming an injection vector.

4. SQL Injection Testing

- **Goal:** Construct a payload to manipulate backend database logic.
- **Actions:** Formulated a target payload (**administrator' --**) designed to comment out the password validation query logic while specifying the **administrator** user account.

5. Verification

- **Goal:** Confirm successful exploitation and assess total business impact.
- **Actions:** Submitted the payload with an arbitrary password. The application logged in directly as the **administrator** user, verifying a complete authentication bypass.

6. Documentation

- **Goal:** Record proof-of-concept evidence and technical findings.
- **Actions:** Documented step-by-step reproduction steps, screenshots, and impact analysis for professional reporting.

4. Finding: SQL Injection Authentication Bypass

Metadata Field	Value
Finding ID	FIND-2026-001
Title	SQL Injection Authentication Bypass
Severity	High
CWE ID	CWE-89 – Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
OWASP Top 10	A03:2021 – Injection

CVSS v3.1 Score	8.8 (High)
Affected Functionality	User Authentication (Login Page)
Affected Parameter	Username
Testing Environment	PortSwigger Web Security Academy – SQL Injection Vulnerability Allowing Login Bypass

Description

During the security assessment, an SQL Injection vulnerability was identified in the application's authentication functionality. Testing demonstrated that the username input field did not adequately handle specially crafted SQL characters. A single quotation mark (') caused an internal server error, indicating that user-supplied input was being processed in a manner consistent with SQL query interpretation.

Further testing using the authentication bypass technique provided in the educational lab successfully authenticated the session as the administrator user without requiring valid administrator credentials. This confirms that the authentication logic can be manipulated through SQL Injection.

The observed behavior indicates that the application does not sufficiently protect user input before it reaches the database query used for authentication. As a result, an attacker may be able to alter the intended logic of the authentication process and gain unauthorized access.

The vulnerability was manually verified within the controlled educational laboratory environment.

5. Root Cause

The assessment indicates that the application's authentication functionality does not adequately validate or safely process user-supplied input before it is used in the authentication process. During testing, specially crafted input submitted through the **username** field affected the application's authentication behavior, demonstrating that malicious input was interpreted as part of the SQL statement rather than being treated as ordinary text.

The vulnerability is consistent with insecure handling of database queries commonly associated with SQL Injection. Based on the observed behavior, the following weaknesses are likely to have contributed to the vulnerability:

- User input is not sufficiently validated before being processed.
- Authentication logic allows SQL metacharacters to influence query execution.
- The application does not appear to use secure query construction techniques, such as parameterized queries or prepared statements.
- Input containing SQL syntax is not safely handled before reaching the database layer.

These weaknesses allow an attacker to manipulate the authentication process and bypass normal credential verification.

6. Attack Scenario

An external attacker accesses the application's login page without possessing valid credentials. During testing, the attacker first establishes the application's normal authentication behavior by submitting invalid login credentials. The attacker then supplies specially crafted SQL input through the username field to determine whether the application improperly processes user-controlled input.

The application's response confirms that SQL syntax influences the authentication mechanism. The attacker subsequently submits an authentication bypass payload provided within the educational lab, resulting in successful authentication as the **administrator** user without knowledge of the legitimate password. This demonstrates that the authentication mechanism can be manipulated through SQL Injection, allowing unauthorized access to privileged functionality.

If exploited in a production environment, this attack could provide unauthorized users with access to privileged administrative functionality.

7. Impact Analysis

Confidentiality

The vulnerability allows unauthorized access to an administrator account. If exploited in a real-world application, an attacker could gain access to sensitive information available to privileged users, including user records, administrative data, configuration information, or other confidential business information.

Impact: High

Integrity

Successful authentication bypass may grant administrative privileges that allow modification of application data. Depending on the permissions assigned to the compromised account, an attacker could alter records, create or remove user accounts, modify system settings, or manipulate business-critical information.

Impact: High

Availability

Although the laboratory exercise demonstrates authentication bypass only, SQL Injection vulnerabilities can potentially be leveraged to perform destructive database operations when appropriate permissions exist. This could result in service disruption, corruption of application data, or loss of system availability.

Impact: Medium to High (depending on database permissions)

Business Impact

If this vulnerability existed in a production environment, it could have significant business consequences, including:

- Unauthorized administrative access to the application.
- Exposure of sensitive customer or organizational data.
- Compromise of the application's authentication mechanism.
- Loss of customer trust and organizational reputation.
- Potential regulatory or compliance violations due to unauthorized data access.
- Potential financial losses associated with incident response, remediation efforts, regulatory penalties, and reputational damage.

8. Proof of Concept (PoC)

Objective

The objective of this Proof of Concept was to verify whether the login functionality was vulnerable to SQL Injection and determine whether authentication could be bypassed without valid administrator credentials.

Preconditions

- User can access the login page.
- Target application is running and reachable via HTTP/HTTPS.
- No prior authentication or session cookie is required to reach the login endpoint.
- Testing is performed within the PortSwigger Web Security Academy lab environment.

Steps to Reproduce

Step	Action	Expected Observation
1	Open the primary login page.	The login form displaying username and password fields is fully rendered.

2	Enter invalid credentials (invaliduser / wrongpassword).	Application returns a standard generic authentication error ("Invalid username or password").
3	Test the username field with a single quote (').	Application returns an unexpected response consistent with improper processing of user input.
4	Submit the SQL Injection payload (administrator'--).	Authentication logic is altered, resulting in successful authentication without valid credentials.
5	Verify logged-in user context.	Session transitions successfully to an authenticated state under the administrator account.

Expected Result

The application should securely validate user input, separate control logic from data via parameterized queries, and reject malicious input. Authentication should succeed only when valid credentials matching the target account are supplied.

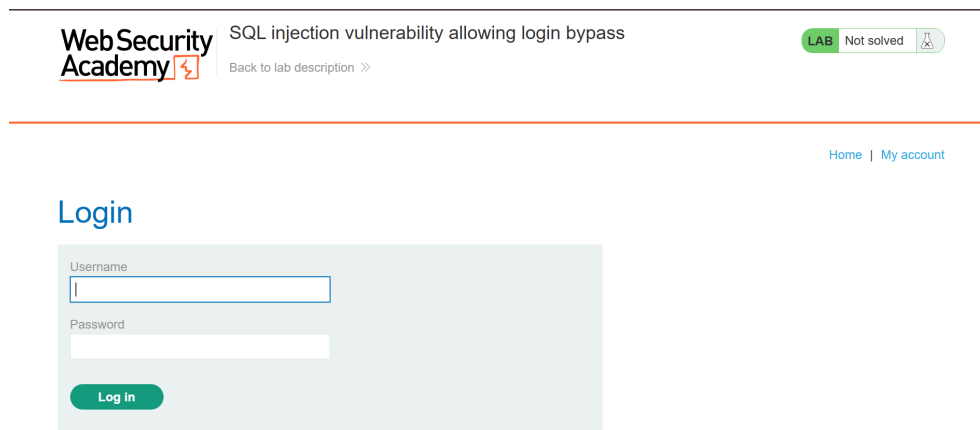
Actual Result

The application processed the injected SQL string directly within the backend query, altering the authentication logic and logging in the session as the **administrator** user without requiring valid administrator credentials.

9.Evidence & Visual Verification

Step-by-Step Evidence Figures

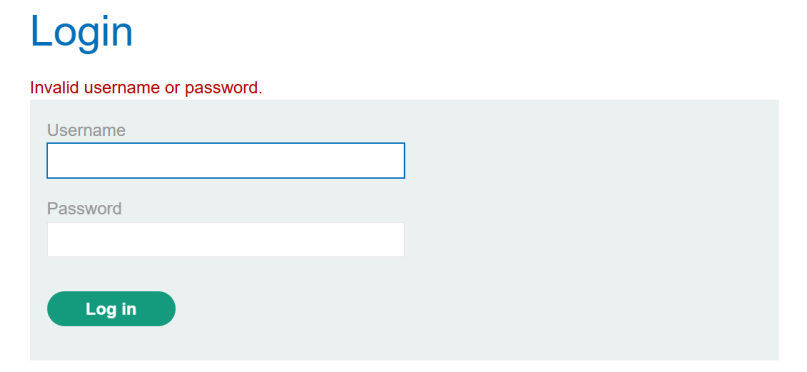
-
- **Figure 1 — Login Page**



The screenshot shows the WebSecurity Academy login page. At the top, the logo 'WebSecurity Academy' is on the left, followed by the text 'SQL injection vulnerability allowing login bypass' and a link 'Back to lab description >>'. On the right, there is a green 'LAB' badge, a 'Not solved' status, and a flag icon. Below the header, there are links for 'Home' and 'My account'. The main content area is titled 'Login' and contains a form with 'Username' and 'Password' input fields and a green 'Log in' button.

- *Observation:* The standard public login portal rendered without requiring initial authentication cookies.

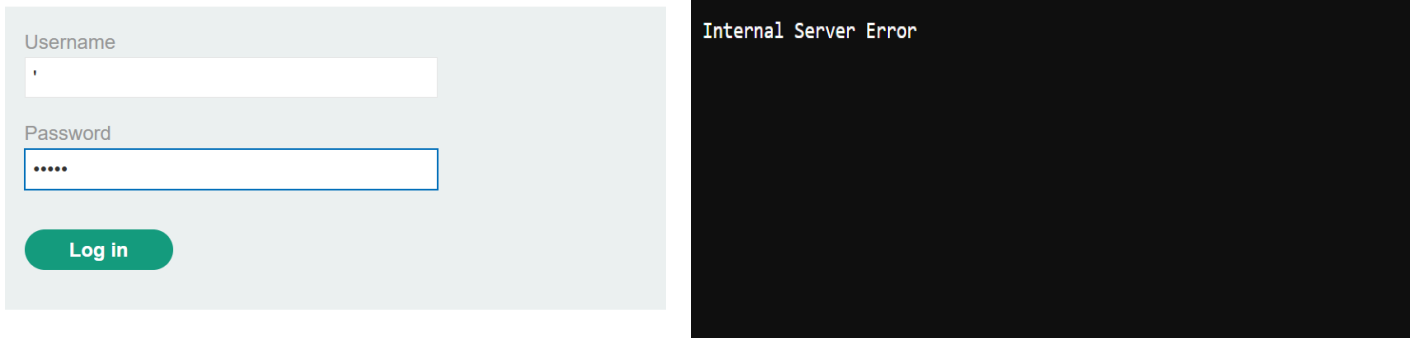
- **Figure 2 — Invalid Login Attempt**



The screenshot shows the WebSecurity Academy login page after an invalid login attempt. The 'Username' and 'Password' input fields are now empty. A red error message 'Invalid username or password.' is displayed above the form. The 'Log in' button remains green.

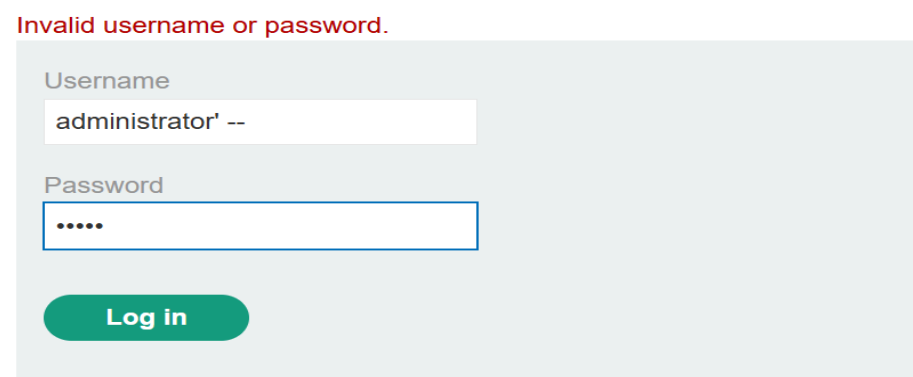
- *Observation:* Entering standard invalid credentials yields a standard **401 Unauthorized** or generic error notification.

- **Figure 3 — SQL Syntax Test**



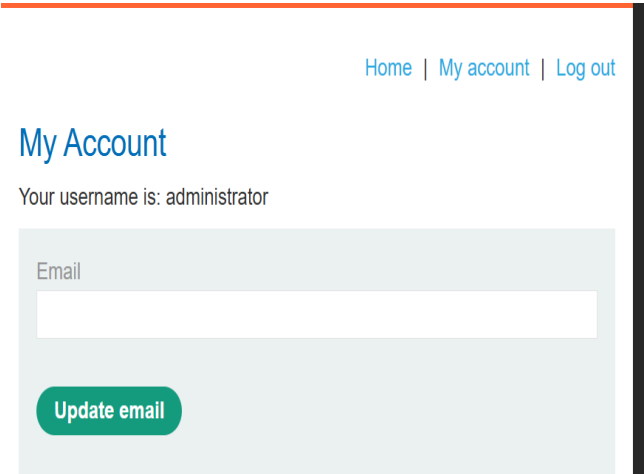
- *Observation:* Submitting **admin'** triggers an unhandled database exception or differential HTTP response, confirming input concatenation.

- **Figure 4 — SQL Injection Payload Submission**



- *Observation:* Submitting **administrator' --** in the **username** parameter suppresses the password verification logic completely.

- **Figure 5 — Successful Administrator Access**



- *Observation:* Successful session creation confirms that the SQL Injection payload altered the authentication logic, granting full administrative privileges.

10. Risk Assessment

Field	Value
Vulnerability	SQL Injection Authentication Bypass
Severity	High

CVSS v3.1 Score	8.8(CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
CWE ID	CWE-89
OWASP Category	A03:2021 – Injection
Attack Complexity	Low
Privileges Required	None
User Interaction	None

Justification

This vulnerability is classified as High Severity (CVSS 8.8) because it requires no privileges and no user interaction, while allowing an unauthenticated remote attacker to completely compromise account governance. By bypassing authentication to obtain an administrative session, the attack results in a High Impact across Confidentiality, Integrity, and Availability, permitting full data exfiltration, system configuration changes, and potential database corruption.

11. Remediation

1. Parameterized Queries (Bound Parameters)

The primary defense against SQL injection is ensuring that all database access utilizes parameterized queries (also known as bound parameters). Parameterization ensures the database treats user input strictly as literal data values rather than executable code structure.

2. Prepared Statements

Developers must replace dynamic string formatting and query concatenation with prepared statements across all database interactions. In a prepared statement, the database compiles the query template first before substituting user values into place, rendering control syntax injection impossible regardless of input content.

3. Server-Side Input Validation

While not a complete defense on its own, robust server-side input validation provides critical defense-in-depth:

- Length: Enforce minimum and maximum length bounds appropriate for user fields.
- Type: Strictly validate data types (e.g., integer, email string) before query execution.
- Format: Use regex pattern matching to ensure input matches expected characters.
- Allowed Characters: Deny control sequences and restrict input to expected alphanumeric character sets where possible.

4. Secure Error Handling

Applications must never expose verbose database engine errors (e.g., MySQL syntax errors or stack traces) directly to end users. Database exceptions must be caught internally and logged securely to monitored audit channels, while presenting generic, uniform authentication error messages ("Invalid username or password") to the client.

5. Principle of Least Privilege

Configure the database service account utilized by the web application with only the minimal set of database privileges required for operation. The application user should not hold administrative privileges (**DBA** or **root**) and must be restricted from executing system commands, reading host files, or dropping operational tables.

6. Continuous Security Testing

To prevent regression and catch similar vulnerabilities before production deployment, organizations should establish:

- Secure Code Reviews: Mandatory peer and static analysis reviews for all database access logic.

- Static Application Security Testing (SAST): Automated static code scanners integrated directly into the CI/CD pipeline.
- Dynamic Security Scans (DAST) & Pen Testing: Scheduled automated scanning alongside routine third-party penetration testing.

Ethical Disclaimer

All testing described in this report was conducted exclusively within the intentionally vulnerable PortSwigger Web Security Academy environment for educational purposes. No testing was performed against unauthorized systems or production environments.

12. References

1. OWASP Top 10 (2021): [A03:2021 – Injection](#)
2. OWASP Prevention Cheat Sheet: [SQL Injection Prevention Cheat Sheet](#)
3. PortSwigger Web Security Academy: [SQL Injection Tutorials & Labs](#)
4. Common Weakness Enumeration: [CWE-89: Improper Neutralization of Special Elements used in an SQL Command](#)
5. FIRST CVSS v3.1 Specification: [Common Vulnerability Scoring System v3.1 Specification](#)

13. Conclusion

The assessment successfully achieved its primary objective by identifying and validating a High-Severity SQL Injection Authentication Bypass vulnerability within the application's login handler.

Demonstrated through execution, an unauthenticated attacker can exploit this weakness using basic SQL syntax control payloads to bypass authentication checks and obtain elevated Administrator access. Due to the low attack complexity and severe impact on

corporate data confidentiality and system integrity, this finding represents a High-Risk exposure. This assessment demonstrates the importance of secure coding practices and continuous security testing in preventing injection-based attacks.

Appendix A: Testing Environment

Parameter	Details
Testing Platform	PortSwigger Web Security Academy -Manual Browser-Based Testing
Browser Environment	Chrome
Operating System	Kali Linux 2026.1
Lab Target Name	PortSwigger Web Security Academy: <i>SQL injection authentication bypass</i>
Testing Date	July 25, 2026

Appendix B: Figure Index

Figure ID	Description
Figure 1	Primary Application Login Page Interface
Figure 2	Standard Invalid Login Credentials Attempt

Figure 3	Single Quote (') SQL Syntax Error Injection Test
Figure 4	SQL Injection Authentication Bypass Payload Submission (administrator'--)
Figure 5	Confirmed Administrator Account Dashboard Access